

PARASCOPIES: WHAT DO LANGUAGE MODELS ACTIVATIONS ENCODE ABOUT FUTURE TEXT?

Nicky Pochinkov

Independent

Dublin, Ireland

`paper@nicky.pro`

Yulia Volkova

Independent

London, UK

Anna Vasileva

Independent

Moskow, Russia

Sai V R Chereddy

Independent

Navi Mumbai, India

ABSTRACT

Interpretability studies in language models often investigate forward-looking representations of activations. However, as language models become capable of doing ever longer time horizon tasks, methods for understanding activations often remain limited to testing specific concepts or tokens. We develop a framework of Residual Stream Decoders as a method of probing model activations for paragraph-scale and document-scale plans. We test several methods and find information can be decoded equivalent to 5+ tokens of future context in small models. These results lay the groundwork for better monitoring of language models and better understanding how they might encode longer-term planning information.

1 THE PLANNING DECODABILITY HYPOTHESIS

Large Language Models (LLMs) generate coherent multi-paragraph text through autoregressive prediction. However, coherence over increasingly long time horizons (Kwa et al., 2025) suggests some degree of forward-thinking in writing. In this paper, rather than asking whether models “plan” in an anthropomorphic sense, we operationalize a test for planning: planning at scale X exists if information about scale- X content is decodable from activations prior to being generated. This planning **Decodability Hypothesis** makes planning empirically testable while remaining agnostic about the underlying mechanisms, yet presents two main potential issues. The first concern is that a sufficiently complex probe may be able to implicitly infer plans that are not actually present, due to correlations, the second is that negative results would not necessarily rule out the existence of plans that our methods could not decode.

We primarily investigate this hypothesis at the paragraph scale outputs, but also briefly investigate outline of full outputs. We choose this scale because some minimal level of planning is likely to be functionally useful, as maintaining topic coherence requires some representation of upcoming content. Additionally, paragraph boundaries (“`\n\n`” tokens) provide natural intervention points for studying information transitions. Our investigation reveals that some information is decodable with relative ease in models as small as Llama 3.2 3B, providing evidence of limited planning.

2 RELATED WORK

Much interpretability work has been focused on understanding the hidden-layer activations of language models. There are various methods ranging from simple Logit Lens analysis (nostalgebraist, 2020), to circuit analysis (Elhage et al., 2021; Olsson et al., 2022; Wang et al., 2022), to concept analysis with Sparse Auto Encoders Cunningham et al. (2023); Team (2023); Gao et al. (2024). Additionally, there is much work on probing models (Belinkov, 2022), predominantly for a single trait Burns et al. (2023) or specific aspect(s) (Tenney et al., 2019; Zou et al., 2023) of the output. Each of these predominantly focuses on understanding the effect of the layer activations on some immediate token or concept.

Recent work has investigated planning mechanisms in language models (Vaswani et al., 2017) across multiple scales and methodological approaches. At the token level, Pal et al. (2023) pioneered probing hidden states to predict future tokens, while Wu et al. (2024) distinguished between “pre-caching”

versus "breadcrumbs" explanations for future-oriented representations. Moving to higher semantic levels, Pochinkov et al. (2024) and Ghandeharioun et al. (2024) introduced limited methods for decoding model activations by transferring them to a new context, and we build upon this work.

Mechanistic circuit analysis has provided some of the strongest evidence for planning. Lindsey et al. (2025) demonstrated "Planning in Poems" through circuit tracing, while their companion work on "Hidden Goals" (Marks et al., 2025) showed that models can represent goal states that guide generation. In specialized domains, Jenner et al. (2024) found evidence of learned look-ahead in chess-playing networks, and Tafteeque et al. (2024) studied planning in block movement games. This paper aims to provide a more general method for studying longer-horizon planning in LLMs.

The relationship between agency and planning has also received attention, with Li et al. (2024) finding that RLHF changes the way models represent future information, leading to more structured generation patterns. Methodologically, work on representation engineering (Zou et al., 2023), activation patching (Turner et al., 2023), and interpretability frameworks like Patchscopes (Ghandeharioun et al., 2024) and LatentQA (Pan et al., 2024), have all developed tools applicable to planning research, without directly focusing on interpretability of longer-term planning.

Additionally, there is a variety of work that tries to modify model training to explicitly plan ahead, either to improve inference speed (Bhendawade et al., 2024), or to try to get the model to write explicit plans (team et al., 2024; Yin et al., 2024). The focus of this paper is instead on understanding how existing transformers might already encode future text information.

3 RESIDUAL STREAM DECODERS AND PARASCOPIES

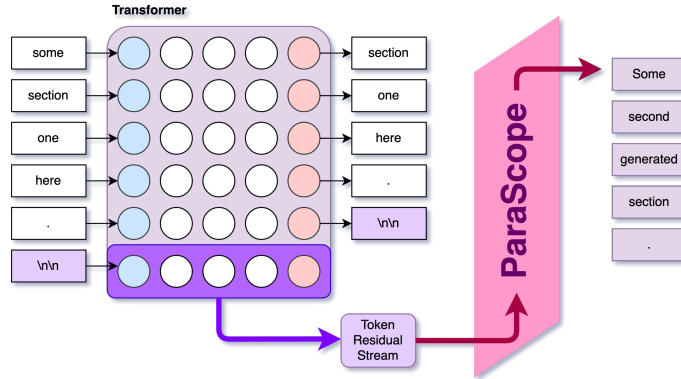


Figure 1: Simple diagram showing the idea behind ParaScopes. The residual stream of an LLM is taken at a specific point, and we try to use ParaScope methods to infer what the LLM might say next.

When language models generate text, they likely need to maintain some form of forward thinking to produce coherent output. While there are many aspects of cognition that one can call "planning", we focus on a specific operationalizable question: Does a language model encode information about its likely future outputs within its internal representations?

To investigate planning, we need a systematic way to extract information from a model's internal representations. We define a **Residual Stream Decoder (RSD)** as any method that can reconstruct future content from current activations better than chance.

Formally, given a transformer's residual stream $R_i \in \mathbb{R}^{L \times d}$ at position i , a decoder consists of:

- An activation extractor that selects relevant information from R_i
- A mapping function that transforms this information into predictions about future content
- An evaluation metric that measures how well these predictions match actual outcomes

The key insight is that if models truly maintain forward-looking information, then decoders trained on this internal state should outperform baselines that lack access to the model's "planning" representations. This information-theoretic perspective allows us to sidestep debates about whether models

truly "plan" in a cognitive sense, instead asking what forward-looking information can be decoded from the model's activations, and use these as methods to test the Decodability Hypothesis.

We additionally coin the term **ParaScope** to be a Residual Stream Decoder specifically used to decode the next paragraph or section of text.

3.1 CORE METHODS AND SETUP

We examine how language models encode information about upcoming paragraphs within their residual stream. Our investigation centers on transition points between paragraphs, marked by "`\n\n`" tokens, where we hypothesize that the model may be encoding information about the upcoming content.

For our experiments, we use Llama-3.2-3B-Instruct (Dubey et al., 2024) as the primary model, with temperature=0.3 for all generation tasks. We additionally employ SONAR models (Duquenne et al., 2023) for text auto-encoding (TAE) tasks, and Qwen 3 embedding model (Zhang et al., 2025) for text vectorization as a method to compare outputs.

For our dataset, we require data that represents the outputs of the LLM so we can understand its thinking process. We use synthetic question prompts based on FineWeb-Edu (Lozhkov et al., 2024), and generate a diverse set of LLM model outputs based on each prompt, up to 1 million text examples with a temperature of 0.3. See Appendix A for more details.

For the purposes of this work, we consider a "paragraph" to be a part of the text that is separated by a "`\n\n`" token. We find, in most cases, that this definition is a natural boundary for the model to parse and allows for the evaluation of planning at the paragraph level.

In order to evaluate how well our extraction of potential future context worked, we also need a way to compare similarity between text outputs. The main methods we use are: 1. traditional methods including cosine similarity from text-embed models (Zhang et al., 2025), and BLEURT-20 (Pu et al., 2021), and 2. LLM-as-a-Judge (Zheng et al., 2023b; Liu et al., 2023; Kim et al., 2024b; or Various, 2025) with rubric scoring. For our LLM rubric, we focus on relevant topics in text, coherence, subject match, entity preservation, and detail preservation. For more details on the rubric and our setup, see Appendix C

3.1.1 RESIDUAL STREAM DECODER METHODS

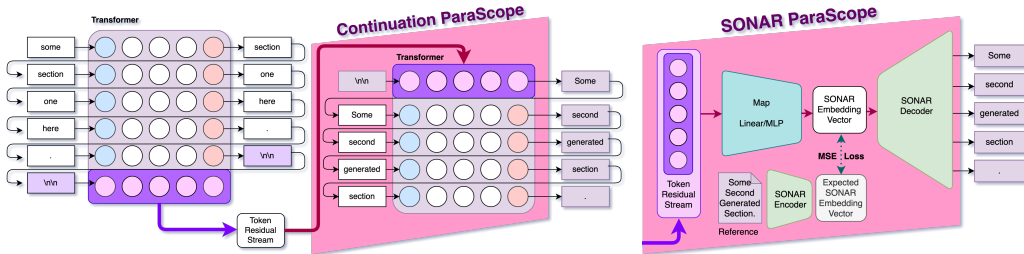


Figure 2: Continuation ParaScope (Left) and TAE ParaScope (Right). The former takes the whole residual stream of the model and passes it into a blank-context copy of the model for decoding. The latter takes the residual stream of the model and trains a map to output a text autoencoder vector.

We introduce two complementary Residual Stream Decoder approaches:

Continuation ParaScope: This method, inspired by Ghandeharioun et al. (2024) and Pochinkov et al. (2024), intervenes directly on model activations to probe what the model might generate. We insert a blank prompt consisting of '`<bos>\n\n`', replace the residual stream activations of the '`\n\n`' token with the activations saved from an original generation, then generate up to 128 tokens. This approach requires no training, instead relying on the model to access any information it may have stored.

TAE ParaScope: the Text AutoEncoder ParaScope learns a mapping from the residual stream to a structured text auto-encoder embedding space from the SONAR model. We normalize the residual stream activations (treating each dimension as independent, using mean and standard deviation normalization) and train a map to predict the TAE embedding vectors of the upcoming paragraph.

We choose to use a linear map, which takes the normalized residuals diffs from some subset of layers, and outputs a SONAR vector of dimension 1024. See Appendix G.1 for details.

The predicted embedding vectors are then decoded using SONAR’s decoder to produce human-readable text. This structured approach allows us to leverage the semantic understanding encoded in the SONAR embedding space.

3.1.2 BASE EVALUATION FRAMEWORK

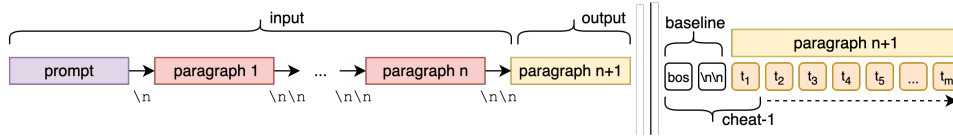


Figure 3: Basic diagram explaining the next-paragraph prediction task (left) and showing how we produce the baseline generation and cheat-k predictions (right)

To evaluate our methods, we must first establish appropriate baselines for comparison.

Random/Blind Baseline: Generation with only a blank ‘<bos>\n\n’ context, providing a lower bound worst-case for performance.

Cheat-K Baseline: Generation with ‘<bos>\n\n’ plus K tokens of the actual upcoming section revealed ($K \in 1, 5, 10$). For fairness, we filter examples where more than 50% of the text would be exposed. In Appendix B we briefly investigate the degree to which a model is able to reproduce a similar text from revealing cheat- K tokens, showing saturation of performance at around 20 tokens.

Regeneration: Taking the whole previous context (prompt + generated sections up until the current section) and generating what could come next, giving us the ground truth we aim to match.

Auto-Decoded: Taking the reference text, encoding it with SONAR, and decoding it again, providing an upper bound on SONAR-based methods.

4 PARAGRAPH-LEVEL PLANNING: EVIDENCE AND STRUCTURE

We first test using traditional methods, including cosine similarity from Qwen 3 Embed 0.6B, and BLEURT score. Both show similar results. For cosine similarity, we find that Continuation ParaScope (mean 0.43) and TAE ParaScope (0.55) both perform significantly better than random baselines (0.20), while being significantly worse than ground truth (regenerated 0.82, auto-decoded 0.94). Both RSD methods are comparable to the cheat-5 baseline (0.50), with the Continuation ParaScope performing slightly worse, while TAE ParaScope performs slightly better.

These seem to suggest that the model is storing some limited plans about the future within its activations, giving evidence for the Decodability Hypothesis, comparable to revealing around five future tokens into the future.

We additionally test for more fine-grained features using ruberic-based LLM-as-a-judge approach. We primarily compare how well general subjects (on a scale from -1 to 4) and fine-grained details (on a scale from -1 to 3) match the baseline generation (see Appendix D for details).

When looking at subject match, we find that TAE ParaScopes generally seem to outperform Continuation ParaScopes. With a score threshold of ≥ 2 corresponding to being in a related general domain or better 76% of TAE ParaScopes achieved this level, compared to 43% for Continuation ParaScopes and 51% for the cheat-10 baseline. This significantly outperforms the random baseline of 4%, while still being short of the 99% achieved by ground-truth regeneration.